

Comparing `left[iLeft]` and `right[iRight]` again, we find that, of 9 and 8, `right[iRight]`'s 8 is smaller. The code appends the 8 to `sortedResult` and increases `iRight` from 0 to 1. Here's the state of the variables now:

```
sortedResult = [2, 5, 8]
    left: [2, 9]    right: [5, 8]
    indices:  0  1          0  1
    iLeft = 1      ^
    iRight = 2          ^
```

Because `iRight` is now 2 and equal to the length of the `right` list, the remaining items in `left` from the `iLeft` index to the end are appended to `sortedResult`, as no more items remain in `right` to compare them to. This leaves `sortedResult` as `[2, 5, 8, 9]`, the sorted list it needs to return. This merging step is performed for every call to `mergeSort()` to produce the final sorted list.

When we run the *mergeSort.py* and *mergeSort.html* programs, the output shows the process of sorting the `[2, 9, 8, 5, 3, 4, 7, 6]` list:

```
.....mergeSort() called on: [2, 9, 8, 5, 3, 4, 7, 6]
.....Split into: [2, 9, 8, 5] and [3, 4, 7, 6]
.....mergeSort() called on: [2, 9, 8, 5]
.....Split into: [2, 9] and [8, 5]
.....mergeSort() called on: [2, 9]
.....Split into: [2] and [9]
.....mergeSort() called on: [2]
.....mergeSort() called on: [9]
The two halves merged into: [2, 9]
.....mergeSort() called on: [8, 5]
.....Split into: [8] and [5]
.....mergeSort() called on: [8]
.....mergeSort() called on: [5]
The two halves merged into: [5, 8]
The two halves merged into: [2, 5, 8, 9]
.....mergeSort() called on: [3, 4, 7, 6]
.....Split into: [3, 4] and [7, 6]
.....mergeSort() called on: [3, 4]
.....Split into: [3] and [4]
.....mergeSort() called on: [3]
.....mergeSort() called on: [4]
The two halves merged into: [3, 4]
.....mergeSort() called on: [7, 6]
.....Split into: [7] and [6]
```